

Normalization

First Normal Form (1NF)

A database satisfies 1NF if it meets three strict conditions:

1. **Atomic Values:** Every column contains only single, indivisible values. There are no comma-separated lists or multi-valued attributes.
2. **Unique Rows:** Each table has a defined Primary Key ensuring all records are completely unique.
3. **Consistent Data Types:** All entries in any given column share the exact same data type.

Examples

- **The Right Way (1NF Compliant):** In our Customers table, if a customer has two phone numbers, we do not store them as "9876543210, 9123456789" in a single cell. Instead, we create an explicit structure or assign a single atomic value per record.
- **The Right Way (1NF Compliant):** In the Order_Items table, rather than storing a single row for an entire basket like Products_Bought = "Prod_01, Prod_02, Prod_03", every single ordered item gets its own independent, atomic row (Order_Item_ID).

Second Normal Form (2NF)

To achieve 2NF, a database must first satisfy 1NF, and it must eliminate **Partial Dependencies**. This means every non-key column must depend on the entire primary key, not just a part of it.

- This rule is only an issue for tables with **Composite Primary Keys** (keys made of two or more columns combined).
- In our design, every single table uses a standalone, single-column primary key (Customer_ID, Product_ID, Order_ID, etc.) configured as an AUTO_INCREMENT column.
- Because our primary keys are single columns, partial dependency is physically impossible. Every non-key column automatically depends on the entire primary key.

Third Normal Form (3NF)

To satisfy 3NF, a database must first be in 2NF, and it must eliminate **Transitive Dependencies**. This means a non-key column cannot depend on another non-key column; every column must depend only on the primary key.

Our schema avoids transitive pitfalls through clean separation:

- **No Customer Data in Orders:** The Orders table only stores Customer_ID. It does not store Customer_Name or City. If it did, Customer_Name would depend on Customer_ID, which depends on Order_ID (a transitive dependency), causing massive data duplication.
- **No Category Data in Products:** The Product table only references Category_ID. It does not store the Category_Name or Category_Description. Those attributes live strictly in the Category table, ensuring that changes to a category's description happen in exactly one place.

Conclusion

Normalizing the **RetailHub Pvt. Ltd.** database layout through Third Normal Form (3NF) solves its business problems and creates a well-normalized transactional database suitable for operational reporting and as a foundation for future analytics.

- **Data Integrity:** Splitting mixed data into clean, specialized tables (Customers, Products, Orders) ensures each fact is stored exactly once. This eliminates data inconsistencies, preventing situations where a customer's phone number or a product's price differs across separate records.
- **No Storage Waste:** Removing redundant columns (such as keeping category descriptions inside product rows) significantly reduces database file sizes, cutting down on cloud hosting and infrastructure costs.
- **Analytics Readiness:** This clean 3NF database acts as an ideal "Single Source of Truth." It provides the accurate operational data needed to feed leadership dashboards, optimize inventory control, and track employee performance without dealing with duplicate or broken records.

While a strict 3NF layout is perfect for managing daily transactions, analytical queries across millions of rows can slow down because of the heavy table joins required. As RetailHub scales, the next step will involve selectively **denormalizing** this clean data structure into a Star Schema (Fact and Dimension tables) inside a data warehouse to ensure fast dashboard performance.